

iter 1 walking skeleton → **iter 2**

골격에서 동작하는 흐름.

대한항공 Skypass 예약 시스템에 **Strategy** 패턴을 더하고,
iter 1에서 비워둔 **State** 전이 5개를 모두 켜습니다.

발표

2026 · 05 · 12

김기석 교수님

팀

A팀 · 김정욱 · 이재호 · 김경동

Team A

코드베이스

KoreanAirReservationDomain

Java 17 · Eclipse · AmaterasUML

양식#1 — 전체 기능 리스트

대한항공 Skypass 예약 시스템 · iter1 → iter2 누적

iter 1 완료

iter 2 신규/활성

iter 3-4 예정

기능군	세부기능	ITER 1	ITER 2	ITER 3	ITER 4
인증·회원	회원 로그인 (Skypass)	✓	↑		
	salted SHA-256 인증 강화		●		
	Guest 본인 확인 (PNR + 이름 + 이메일)		●		
검색·예약	항공편 검색 (편도)	✓	✓		
	예약 생성 (단일 segment)	✓	✓		
	승객 정보 입력	✓	✓		
	좌석 자동 배정 → 좌석 맵 선택	✓	↑		
	결제 (PG 송금)	✓	✓		
	예약 확정 + 운임 검증	✓	✓		
발권	e-Ticket 발급 (KE-yyMMdd-NNNN)		●		
	e-Ticket PDF export				○
조회	회원 예약 이력 조회		●		
	Guest PNR 단건 조회		●		
취소·환불	예약 취소 요청 + 확정		●		
	환불 정책 자동 해석 (Strategy)		●		
	PG 환불 송금		●		
마일리지	마일리지 적립 (확정 시)				○

양식#2 — 확장기능 정리표

확장 가능성 있는 기능만 · 직전 대비 신규는 빨간색

● **신규** = iter 1 발표 대비 추가된 확장 항목

기본 기능	확장기능 (추가/변경 가능성)	비고 (예상 REFACTORING/DP, 예상 구현 시점)
예약 취소 + 환불	● fare class 별 환불 정책 자동 해석 · ● PG 송금 연계	Strategy RefundPolicy family · iter 2 적용 완료 · DP#2
예약 조회	회원 예약 이력 + ● Guest PNR 단건 조회	Composite verify (PNR + 이름 + 이메일) · iter 2 · 5회 실패 → 15분 lock
결제·인증	● salted SHA-256 + per-member salt	iter 1 평문 비교 → iter 2 보강 · 별도 DP 적용 없음 (refactoring only)
발권	● e-Ticket 발급 + 좌석 배정 연계	Ticket.generate() 정적 팩토리 · KE-yyMMdd-NNNN · iter 2
좌석 선택	자동 → ● 좌석 맵 사용자 선택 + Hold	SeatInventory + SeatAssignment · iter 2 · iter 3에 Hold timeout Observer 예정
flight status 변동	flight 상태 변경 시 모든 예약에 자동 통보	예상 DP — Observer · iter 3 도입 예정
결제 수단	카드 / 계좌이체 / 마일리지 / 모바일페이	예상 DP — Factory Method (PaymentMethodFactory) · iter 4 도입 예정
환승 항공편	1 itinerary가 다중 segment 합성	기존 Itinerary-Segment 관계 활용 · iter 3 본격 활성화 + GDS interline
앱 설정	전역 통화 / 언어 / 환경 설정	예상 DP — Singleton (AppSettings) · iter 4 도입 예정

양식#3 — Iteration별 Refactoring / DP

총 Refactoring 2회 · DP 2개 적용 (iter 2 기준)

PDF 별첨#1 양식 준수

Iteration	Sub-Iter	Refactoring / Design Pattern	적용처 / 내용 요약 / 기술된 페이지
1st	—	Refactoring#1 WALKING SKELETON + STATE 골격 설계	<code>ReservationState</code> abstract + 8개 구상 클래스 정의 (3개 활성화, 5개 stub). DP#1 도입을 위한 사전 구조. 발표자료 pp.7,9 · 보고서 pp.18-22
	—	DP#1 — State BEHAVIORAL · 상태 캡슐화	<code>Reservation.currentState</code> 가 행동을 위임. 상태별 허용 액션 분리. iter 1 활성화: Initiated → PendingPayment → Confirmed. 발표자료 pp.13 · 보고서 pp.24-28
2nd	2-1	DP#1 — State 보완 전이 5개 본문 채움	iter 1 stub 5개 (Ticketed / CancellationRequested / Cancelled / RefundRequested / Refunded)에 실제 전이 본문 구현. 8 / 8 활성화. 발표자료 pp.13 · 보고서 pp.24-32
	2-1	Refactoring#2 DP#2 도입 사전 인터페이스 정의	<code>RefundPolicy</code> 인터페이스 도입. <code>FareRule.checkRefundPolicy()</code> 가 fare class → policy 매핑. 발표자료 pp.11-12 · 보고서 pp.30-34
	2-2	DP#2 — Strategy BEHAVIORAL · 알고리즘 가족 캡슐화	환불 정책 분기를 <code>RefundPolicy</code> family로 위임. Full / Partial / No 3개 구상. <code>RefundHandler</code> 는 fare class를 모름. 발표자료 pp.11-12 · 보고서 pp.34-40
3rd	—	DP#3 — Observer (예정) BEHAVIORAL	좌석 hold timer 만료 + flight status 브로드캐스트. 예상 발표자료 pp.- · 보고서 pp.-
final	4-1	DP#4 — Singleton (예정)	<code>AppSettings</code> 전역 단일 인스턴스. 채점은 1개만 반영 — 도입 신중.
	4-2	DP#5 — Factory Method (예정)	<code>PaymentMethodFactory</code> 결제 수단 다양화 대비.

한 사람당 두 모듈씩.

패턴 일관성을 위해 정욱이 4개 패턴 모두 담당. 도메인은 균등 분배.

정

김정욱

패턴 + 인프라

ITER 1 (완료)

State

 8 클래스 + Reservation Context

Walking Skeleton + Swing app shell

• ITER 2 (이번)

Strategy

 ·

RefundPolicy

family

RefundHandler

 오케스트레이션

ITER 3 · 4 (예정)

Observer

 ·

Singleton

통합 + AppSettings

재

이재호

검색·좌석·라우팅

ITER 1 (완료)

AuthService

 평문 인증

FlightSearchService

 · Itinerary · Segment

• ITER 2 (이번)

좌석 맵 (

SeatInventory

 ·

SeatAssignment

)

SeatSelectionPanel

Swing UI

ITER 3 · 4 (예정)

환승 + 다도시 (Itinerary 다중 segment)

Admin Dashboard + Factory Method

경

김경동

결제·취소·조회

ITER 1 (완료)

PaymentProcessor

 ·

FareRule

BookingController

 +

ReservationUI

• ITER 2 (이번)

예약 조회 — 회원 + Guest 3중 검증

취소 흐름 +

ReservationLookupService

ITER 3 · 4 (예정)

마일리지 적립 / 차감 · Skypass 연동

e-Ticket PDF · Peer Review 문서

Use Case Diagram

iter 1 vs iter 2

iter 1 7 UC · 4 actors

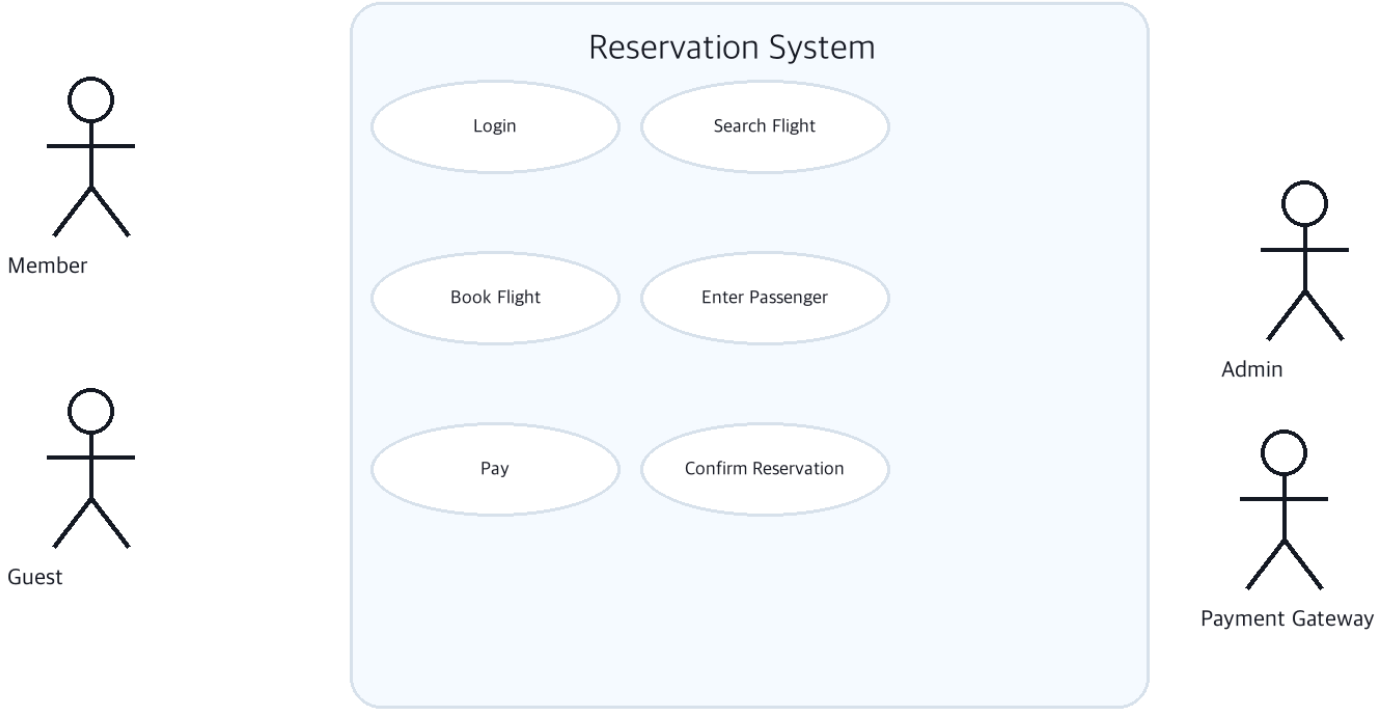
iter 2 17 UC · 7 actors · +10

iter 1

walking skeleton

Use Case Diagram – iter 1

Korean Air Skypass Reservation System



iter 2

+ cancel · refund · lookup · admin

Use Case Diagram – iter 2

Korean Air Skypass Reservation System



UC 시나리오 — iter 2 신규

조회에서 시작해 발권 · 취소 · 환불로 이어지는 4개 시나리오

iter 1 시나리오는 보고서 누적

UC-11

Guest · AuthService · LookupService

Retrieve Booking by PNR

전제 Guest 보유 정보 = PNR + 본인 이름 + 이메일

- 기본 흐름
- 1 Guest가 PNR + 이름 + 이메일 입력
 - 2 `AuthService.verifyGuest(...)` 3중 검증
 - 3 `ReservationLookupService.findByGuestPnr(...)`
 - 4 예약 스냅샷 반환 후 UI에서 취소/환불 진입

대안

A1. 검증 실패 → `"INVALID_CREDENTIALS"` · 디테일 미노출

A2. 회원 조회는 `findByMember(member)` 로 예약 이력 반환

UC-08

Member · Reservation

Cancel Booking

전제 예약 상태 = Confirmed 또는 Ticketed · 예약 조회 완료

- 기본 흐름
- 1 조회 화면에서 취소 대상 예약 선택
 - 2 `processCancellation(pnr)` 호출
 - 3 Reservation → CancellationRequested 전이
 - 4 fare rule 검증 후 Cancelled 전이

대안

A1. 환불 불가 fare class → **Refund Denied** 분기로 이어짐

UC-09

System · Reservation

Issue e-Ticket

전제 예약 상태 = Confirmed · 좌석 배정 완료

- 기본 흐름
- 1 Reservation이 issueTicket() 호출
 - 2 `Ticket.generate(reservation, passenger, seat)`
 - 3 티켓 번호 `KE-yyMMdd-NNNN` 부여
 - 4 Reservation → Ticketed 전이

사후 Reservation에 Ticket 객체 추가, 발권 완료

UC-10

Member · RefundHandler · PG

Process Refund

전제 예약 상태 = Cancelled · fare rule 환불 가능

- 기본 흐름
- 1 Reservation → RefundRequested 전이
 - 2 `RefundHandler.evaluateRefund(pnr, fareClass)`
 - 3 `resolvePolicy(fareRule)` Strategy 해석
 - 4 `policy.calculateRefundAmount(paid)` 위임
 - 5 PG sendRefund + Reservation → Refunded

대안

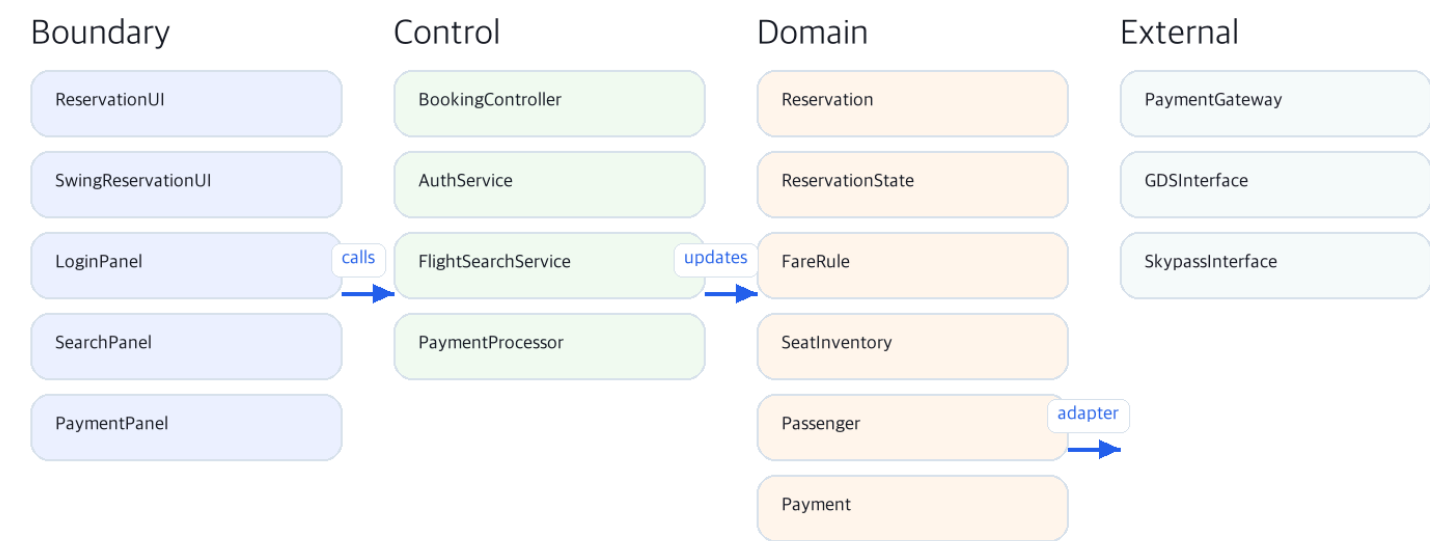
A1. fare class V/O → **NoRefundPolicy** · 0원 반환

iter 1

walking skeleton + State 골격

Class Diagram – iter 1

ECB package overview and pattern families

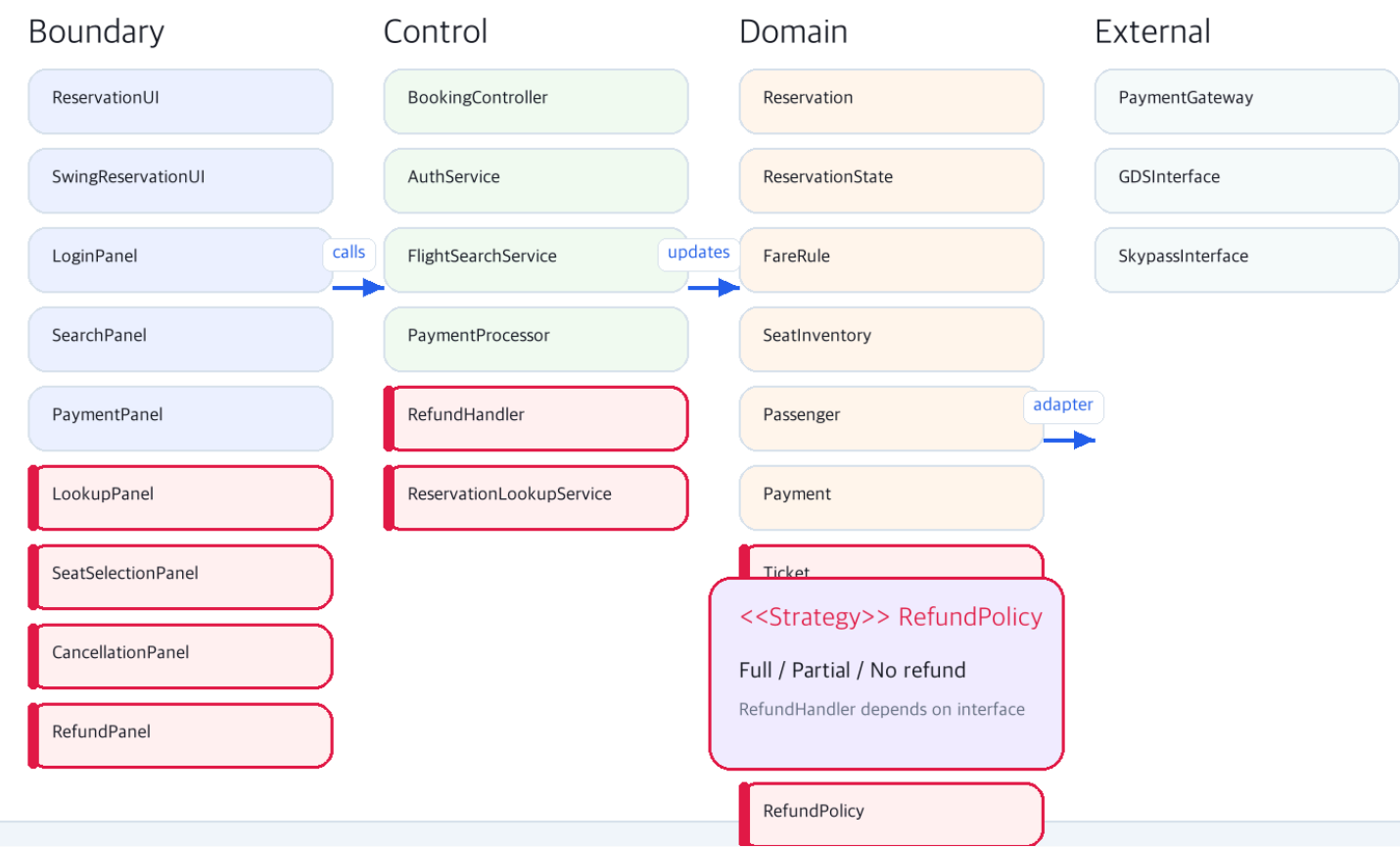


iter 2

+ Strategy · RefundHandler · Lookup · Ticket

Class Diagram – iter 2

ECB package overview and pattern families



중요 클래스 · 메서드

iter 2 신규/변경 9개 클래스 · brief description

INTERFACE · NEW

RefundPolicy

환불 알고리즘 가족 추상화. fare class별 구상 클래스 3개로 분기.

- calculateRefundAmount(int paid)
- getRefundType() : String

ENTITY · NEW

FullRefundPolicy

Y class 전액 환불. paid를 그대로 반환.

- calculateRefundAmount(paid) → paid

ENTITY · NEW

PartialRefundPolicy

M / B class 부분 환불. 위약금 차감.

- calculateRefundAmount(paid) → paid - penalty

ENTITY · NEW

NoRefundPolicy

V / O class 환불 불가. 0원 반환.

- calculateRefundAmount(paid) → 0

CONTROL · NEW

RefundHandler

환불 오케스트레이션. policy 해석 + PG 송금 + state 전이 트리거.

- evaluateRefund(pnr, fareClass)
- resolvePolicy(fareRule) → RefundPolicy
- processRefund(requestId, amount)
- denyRefund(requestId, reason)

CONTROL · NEW

ReservationLookupService

예약 조회 분리. 회원 이력 + Guest PNR 단건.

- findByMember(Member)
- findByGuestPnr(pnr, name, email)

ENTITY · NEW

Ticket

e-Ticket 도메인. KE-yyMMdd-NNNN 식별자.

- generate(reservation, passenger, seat)
- issue() · cancel()

ENTITY · NEW

Refund · RefundRequest

환불 트랜잭션 결과 + 대기열 항목. 상태·금액·요청자 보유.

- Refund-yyMM-NNN id
- RefundStatus enum

CONTROL · MOD

AuthService

salted SHA-256 + Guest 3중 검증 추가. iter 1 평문 비교 대체.

- loginWithHash(num, pw)
- verifyGuest(pnr, name, email)

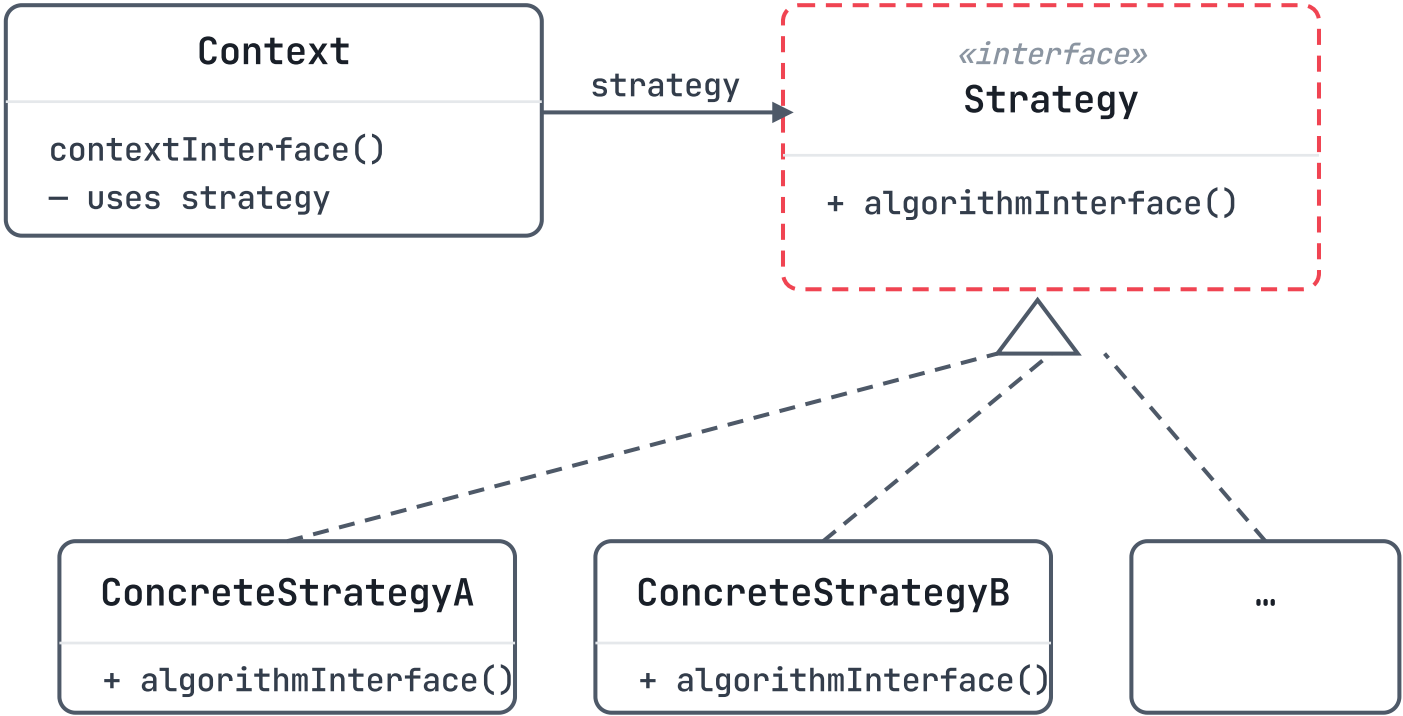
DP#2 Strategy — 교과서 vs 우리 팀

RefundPolicy family는 GoF 원형 그대로

PDF 별첨#2 §1-1 가이드

교과서 · GoF Strategy

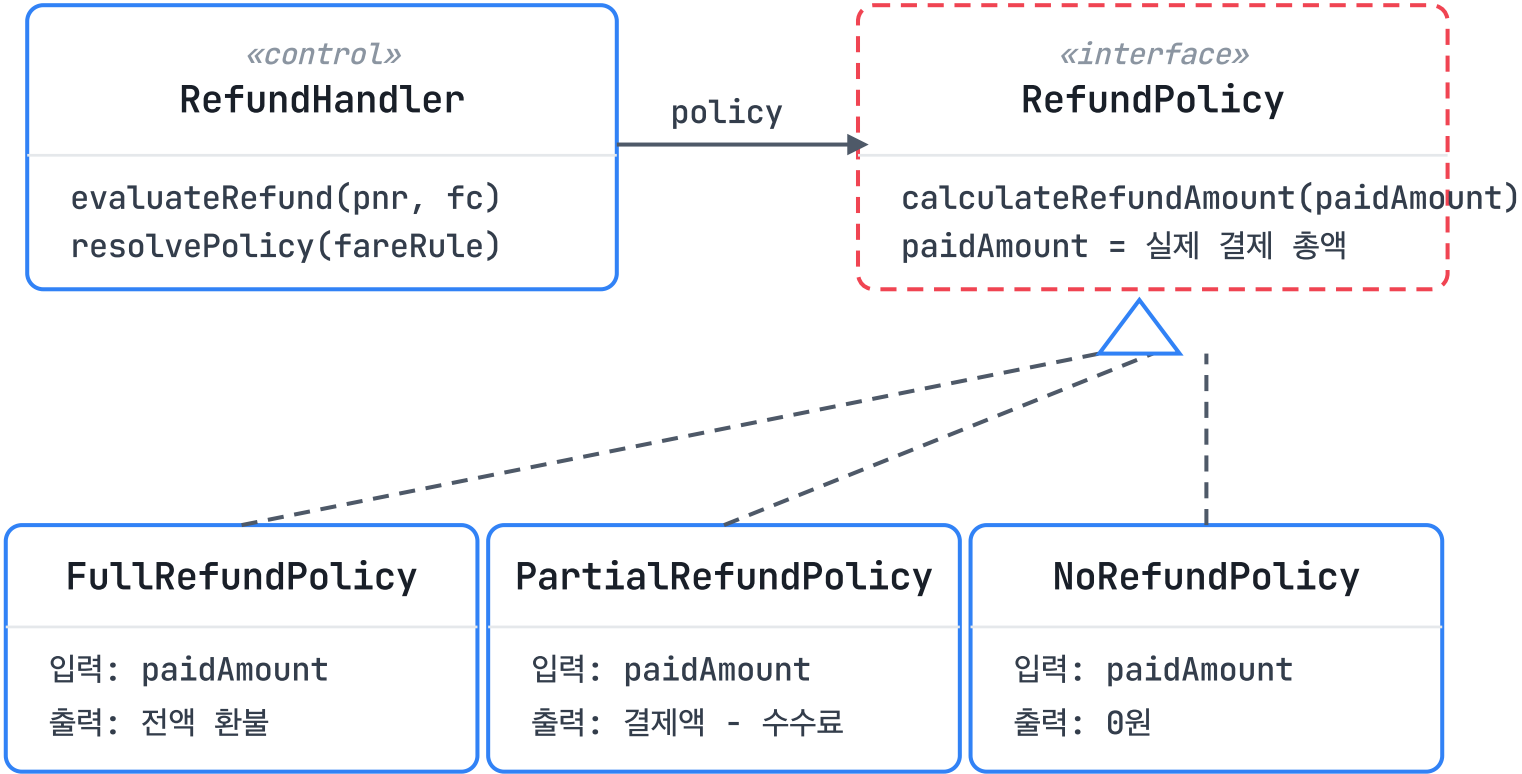
Design Patterns (Gamma et al.) · ch.5



Context는 Strategy의 인터페이스만 알고,
알고리즘 종류는 런타임에 다형성으로 결정

우리 팀 · RefundPolicy family

KoreanAirReservationDomain · iter 2



Y/B → 전액 · M/할인 → 부분 · V/0/불가 → 0원
paidAmount는 Payment.amount 합계, penalty는 취소 수수료

«interface» 구상 클래스 우리 팀 적용 클래스

paidAmount = 결제 총액 · penalty = 취소 수수료 · Context = RefundHandler

Before · iter 1 가정 코드

Strategy 없이 짰다면

```
// RefundHandler - switch 분기 폭발
public int calculateRefund(Reservation r) {
    int base = r.getPaidAmount();
    String fc = r.getFareRule().getFareClass();

    switch (fc) {
        case "Y": return base;
        case "M":
        case "B": return base - 50000;
        case "V":
        case "O": return 0;
        default: throw new
            IllegalArgumentException(fc);
    }
}
```

매
핑

RefundHandler 안
에

fare class 분
기

직접 보유 — 교과서 구성도
의

Context

에 알고리즘이 박혀있는 안티패
턴

After · iter 2 실제 코드

RefundHandler.java · evaluateRefund()

```
// RefundHandler - Context: 정책 선택 후 계산 위임
private RefundPolicy resolvePolicy(FareRule rule) {
    if (rule == null || !rule.isRefundable()) return new NoRefundPolicy();
    String fc = rule.getFareClass();
    if ("Y".equals(fc) || "B".equals(fc)) return new FullRefundPolicy();
    return new PartialRefundPolicy();
}

RefundPolicy policy = resolvePolicy(fareRule);
BigDecimal amount = policy.calculateRefundAmount(paid);
```

FullRefundPolicy

```
calculate(paid) {
    return paid;
}
// Y / B fare
```

PartialRefundPolicy

```
calculate(paid) {
    return paid - penalty;
}
// 할인 운임
```

NoRefundPolicy

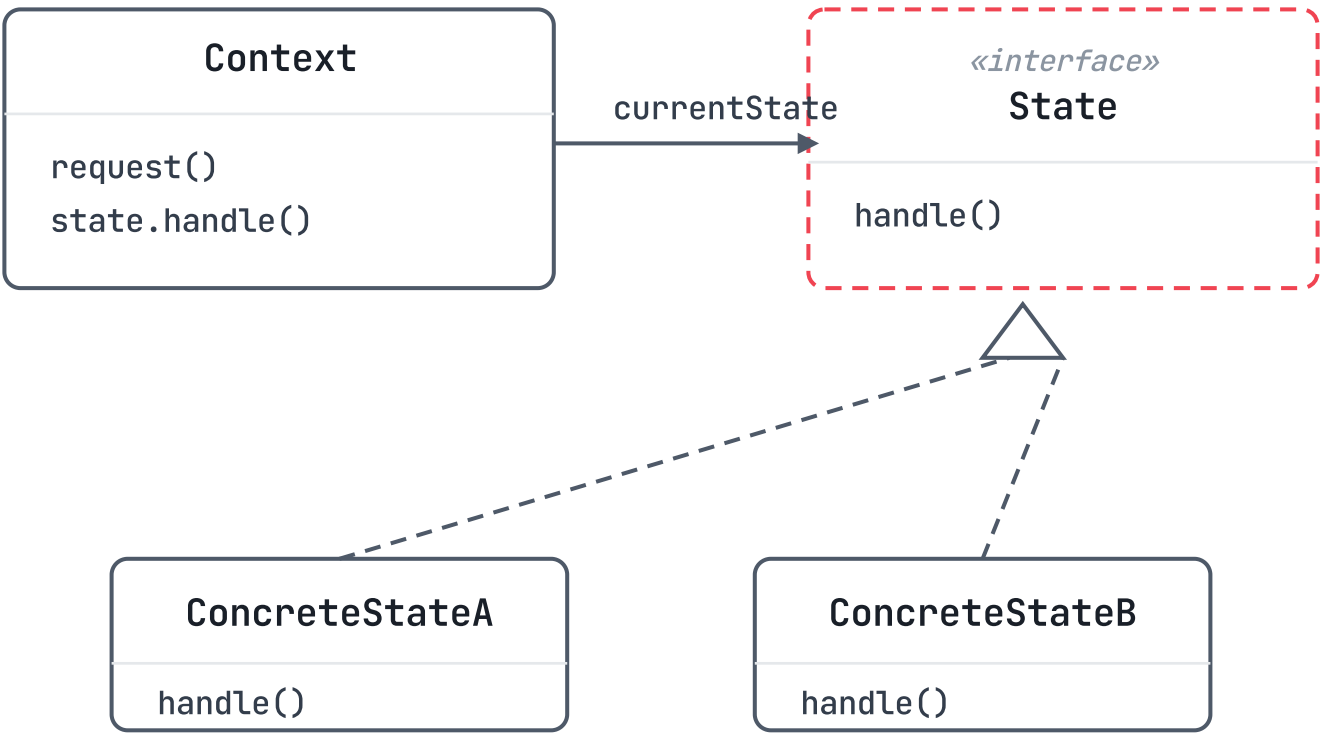
```
calculate(paid) {
    return 0;
}
// 환불 불가
```

NEXT DP resolvePolicy()의 if 분기는 다음 iteration에서 RefundPolicyFactory / Factory Method로 분리할 후보.

매핑 policy.calculateRefundAmount() 호출 = 교과서 구성도의 Context → Strategy.algorithmInterface() 위임

교과서 · GoF State

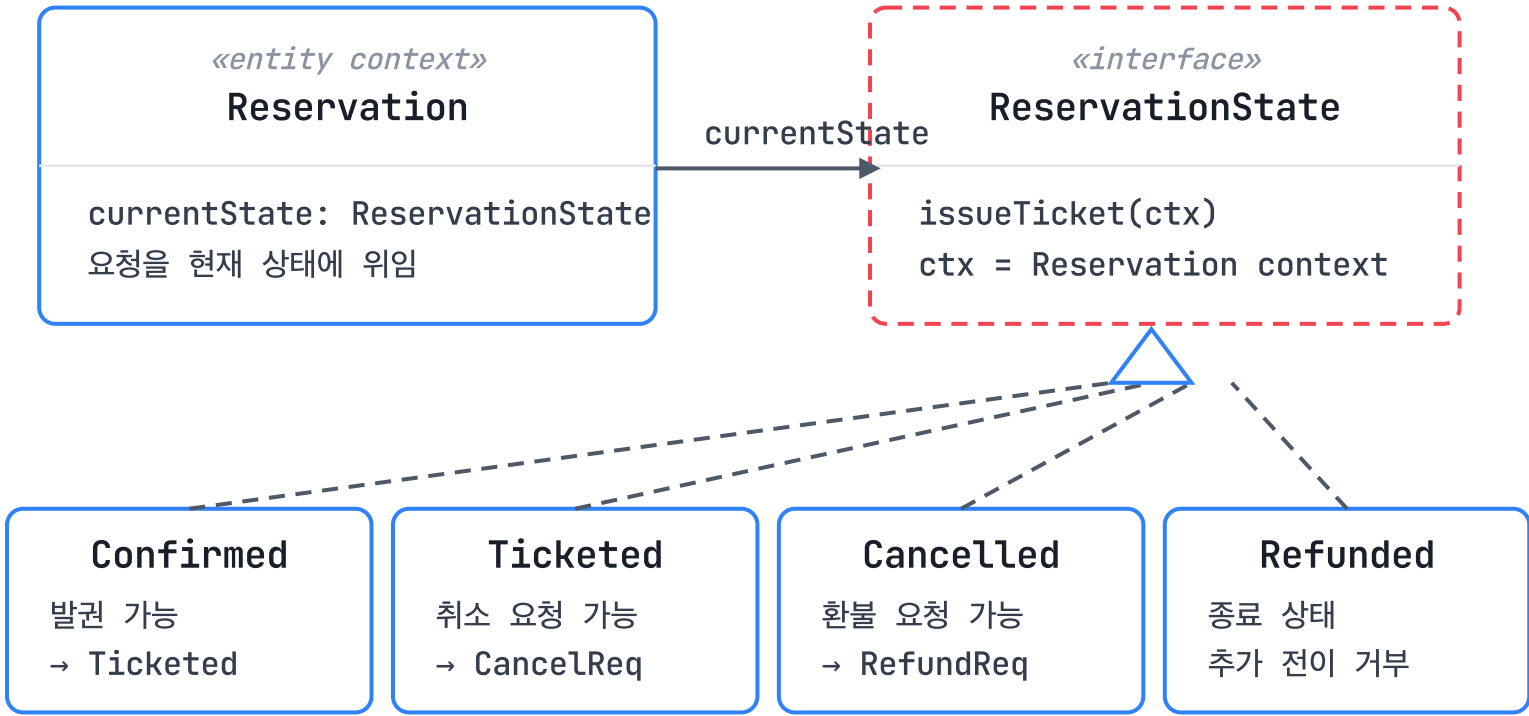
Context가 현재 State 객체에 행동 위임



상태별 행동을 State 구현체로 분리하고,
Context는 현재 상태 객체에 요청을 위임한다.

우리 팀 · ReservationState family

KoreanAirReservationDomain · iter 1 → iter 2



메서드 인자 ctx는 상태가 바뀌는 실제 Reservation 객체
iter 2: 각 concrete state가 허용 행동과 다음 상태를 결정

용어: **ctx** = Reservation context. ConcreteState 메서드가 ctx.setState(...)로 다음 상태를 넣는다. 예: ConfirmedState.issueTicket(ctx) → TicketedState.

Before · iter 1 stub

클래스는 있으나 행동은 미활성

```
// AbstractReservationState.java
public void issueTicket(Reservation ctx) {
    invalid("issueTicket");
}

public void requestCancellation(Reservation ctx) {
    invalid("requestCancellation");
}

public void requestRefund(Reservation ctx) {
    invalid("requestRefund");
}

// iter 1 의미
// State 클래스는 준비됐지만
// 발권·취소·환불 전이는 호출 시 예외
```

교과서 매핑: ConcreteState가 아직 handle()을 구현하지 않은 상태라, Context는 위임할 수 있지만 실제 행동은 거부된다.

After · iter 2 concrete behavior

ConfirmedState / CancelledState 등

```
// ConfirmedState.java
public void issueTicket(Reservation ctx) {
    for (Passenger p : ctx.getPassengers()) {
        Ticket t = Ticket.generate(ctx, p, null);
        ctx.addTicket(t);
    }
    ctx.setState(new TicketedState());
    ctx.updateStatus(ReservationStatus.TICKETED);
}

public void requestCancellation(Reservation ctx) {
    ctx.setState(new CancellationRequestedState());
    ctx.updateStatus(ReservationStatus
        .CANCELLATION_REQUESTED);
}
```

ctx.setState(new TicketedState())가 교과서 State 전이 지점. iter 2에서 5개 전이가 이 방식으로 실제 구현됐다.

State Diagram (Reservation)

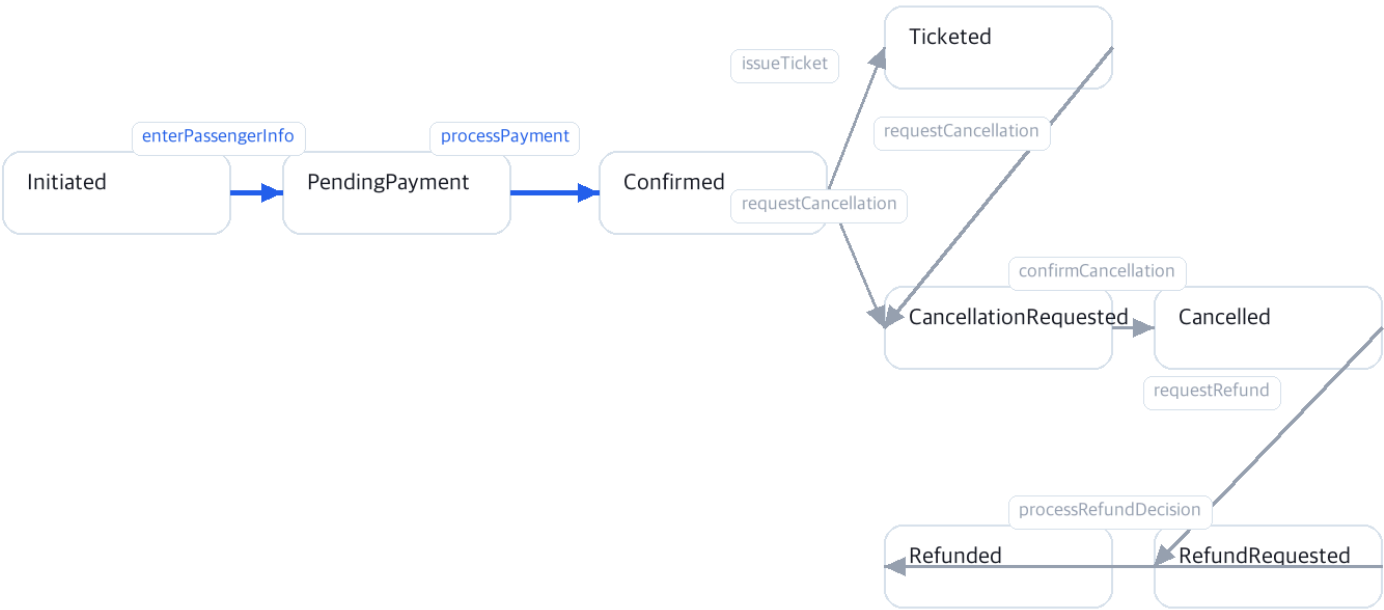
iter 1 vs iter 2 — sub iteration State 활성

iter 1

3 transitions live · 5 stub

Reservation State Diagram - iter 1

State pattern transition coverage

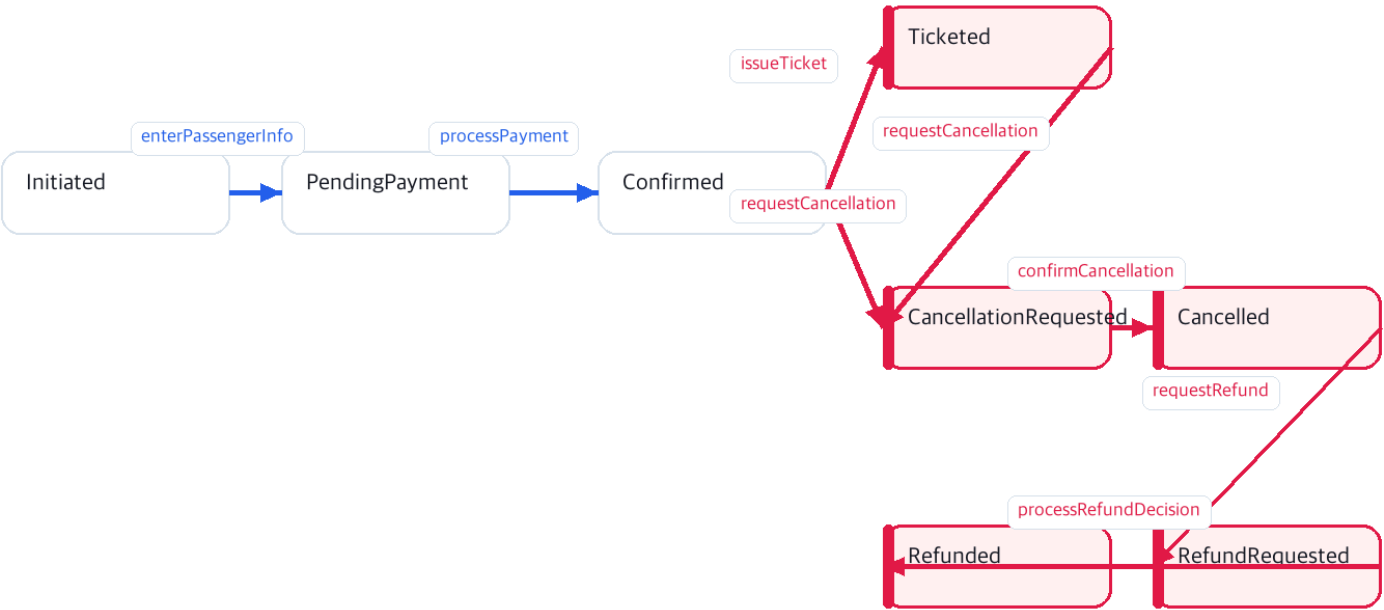


iter 2

+5 transitions activated · 8 / 8 live

Reservation State Diagram - iter 2

State pattern transition coverage



Sequence Diagram — iter 2 신규

cancelRefund · lookupReservation 두 시나리오

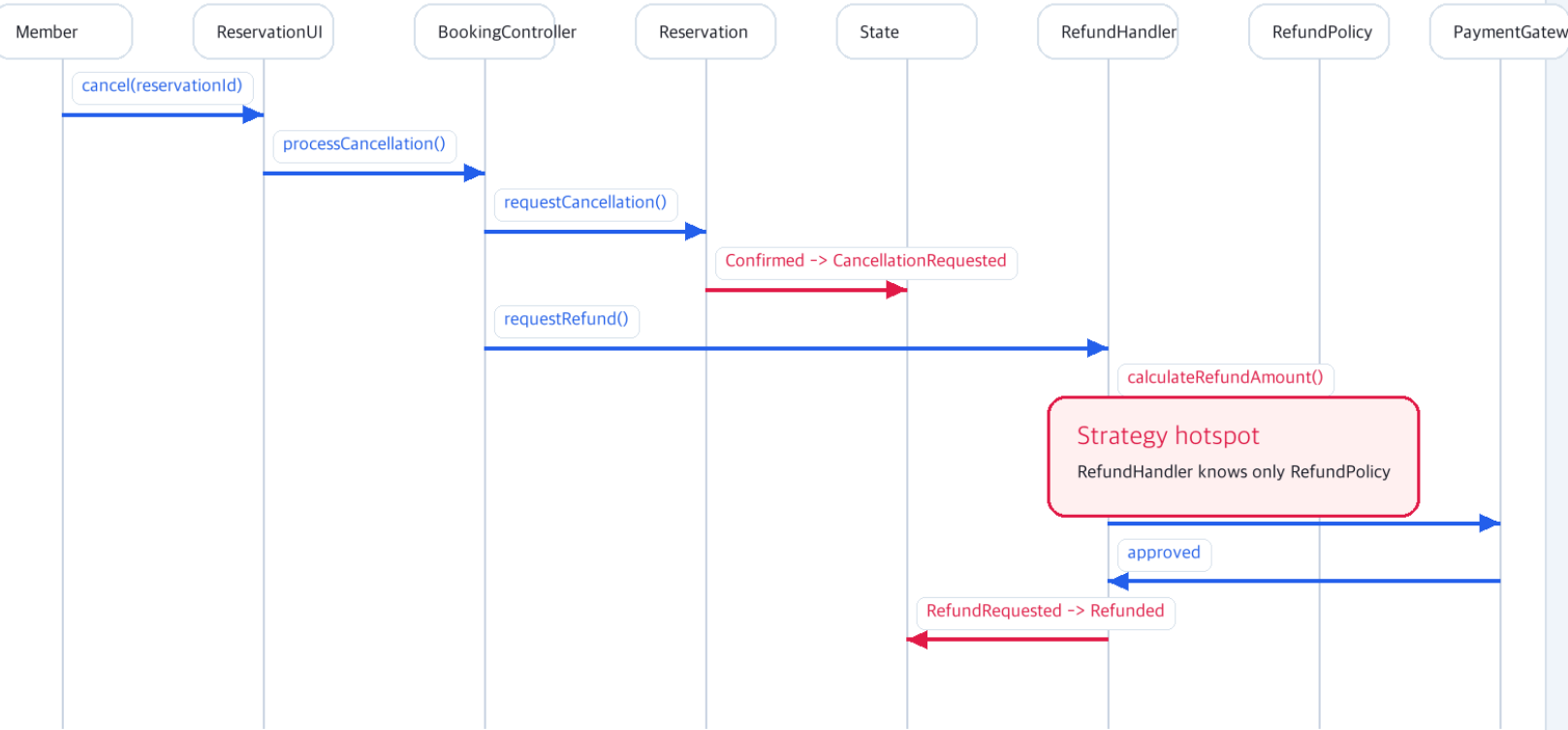
기존 bookFlight / memberBookingTicket / adminOperations은 보고서 누적

cancelRefund-iter2

8 lifelines · 14 messages

Sequence - Cancel + Refund

Strategy delegation point is highlighted in red

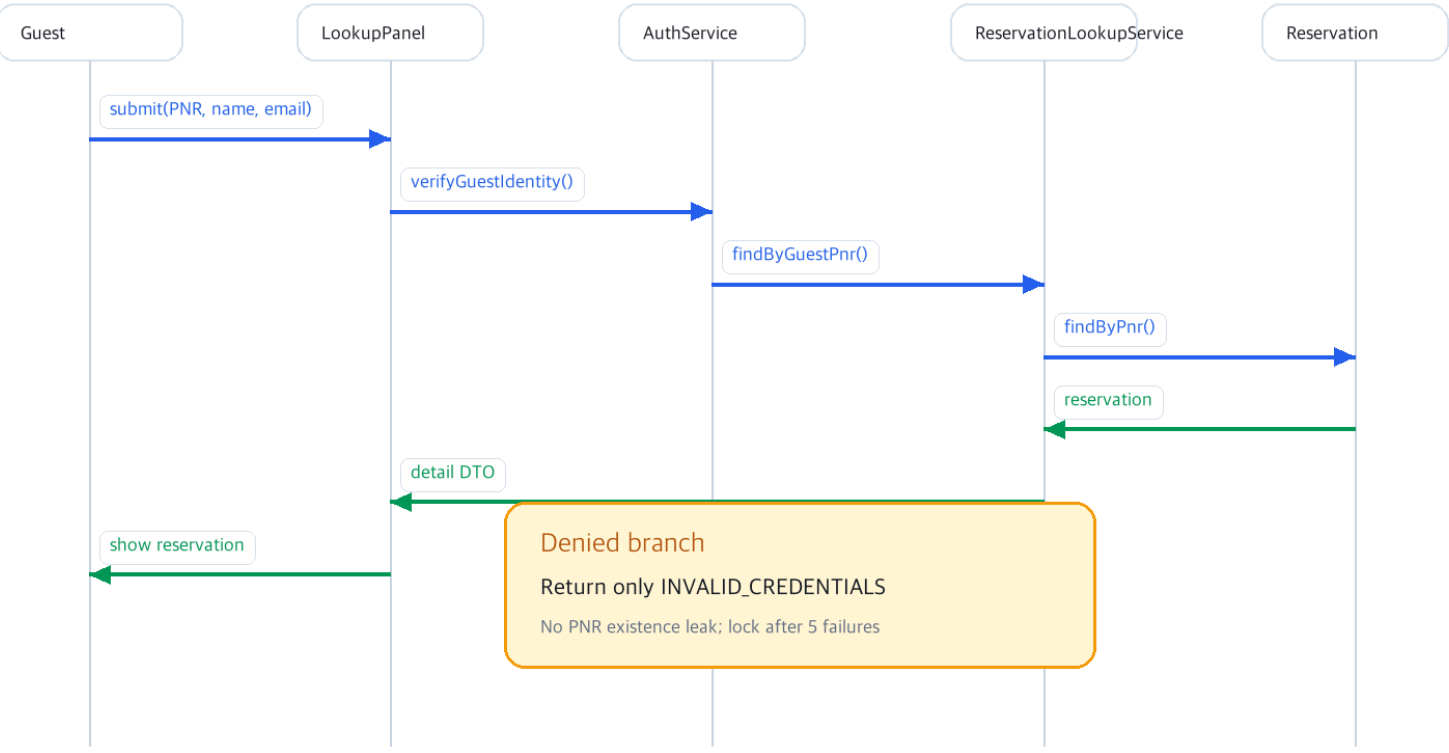


lookupReservation-iter2

6 lifelines · 8 messages

Sequence - Guest PNR Lookup

3~factor verification: PNR + name + email



실행 화면 데모

Swing app + terminal log · UI 조작과 State/Strategy 로그 동시 확인

java -cp bin ...SwingApp

step 1

Lookup

PNR 조회 · Member · Guest 탭

LookupPanel

PNR member/guest lookup

PNR

KE26A7

Guest verify

name + email OK

Result

Reservation detail loaded

step 2

Seat Selection

30×6 좌석 맵 + 선택

SeatSelectionPanel

Seat map selection

Cabin

Economy

Selected

12A

Inventory

HOLD -> ASSIGNED

step 3

Cancellation

사유 입력 + 환불 미리보기

CancellationPanel

Cancel request + preview

Status

Ticketed

Reason

Schedule changed

Preview

FullRefundPolicy

step 4

Refund

Strategy 해석 + PG 송금 진행

RefundPanel

Refund execution

Policy

FullRefundPolicy

Gateway

MockPaymentGateway OK

Status

Refunded

console

State 전이 로그

[STATE] Confirmed → ... → Refunded

Console

State transition backup

[STATE] Confirmed -> Ticketed

[STATE] Ticketed -> CancellationRequested

[STATE] CancellationRequested -> Cancelled

[STATE] Cancelled -> RefundRequested

[STATE] RefundRequested -> Refunded

strategy

정책 해석 로그

[STRATEGY] Y → FullRefundPolicy

Console

Strategy policy backup

[STRATEGY] fareClass=Y -> FullRefundPolicy

[REFUND] paid=421000 penalty=0 amount=421000

[PG] sendRefund(originalPaymentId, amount)

[RESULT] refundStatus=APPROVED

• iter 2 발표 종료

감사합니다.

Thank you.

질문 받겠습니다.

TEAM A

김정욱 · 이재호 · 김경동

ECE 312 · Spring 2026 · 17 / 17